

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №3-4
«Функциональные возможности языка Python»

Выполнил:
Студент группы ИУ5-31Б
Горячева С.А.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Задание:

Задание лабораторной работы состоит из решения нескольких задач.

Файлы, содержащие решения отдельных задач, должны располагаться в пакете lab_python_fr. Решение каждой задачи должно располагаться в отдельном файле.

При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

Задача 1 (файл field.py)

Необходимо реализовать генератор field. Генератор field последовательно выдает значения ключей словаря.

Пример:

```
goods = [
    {'title': 'Ковер', 'price': 2000, 'color': 'green'},
    {'title': 'Диван для отдыха', 'color': 'black'}
]
```

field(goods, 'title') должен выдавать 'Ковер', 'Диван для отдыха'

field(goods, 'title', 'price') должен выдавать {'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха'}

- В качестве первого аргумента генератор принимает список словарей, дальше через *args генератор принимает неограниченное количество аргументов.
- Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно None, то элемент пропускается.
- Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно None, то оно пропускается. Если все поля содержат значения None, то пропускается элемент целиком.

Шаблон для реализации генератора:

```
# Пример:
# goods = [
#     {'title': 'Ковер', 'price': 2000, 'color': 'green'},
#     {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'}
# ]
# field(goods, 'title') должен выдавать 'Ковер', 'Диван для отдыха'
# field(goods, 'title', 'price') должен выдавать {'title': 'Ковер', 'price': 2000}, {'title': 'Диван для

def field(items, *args):
    assert len(args) > 0
    # Необходимо реализовать генератор
```

Листинг кода

```
1 goods = [
2     {'title': 'Ковер', 'price': 2000, 'color': 'green'},
3     {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'}
4 ]
5
6 def field(items, *args):
7     assert len(args) > 0
8     if len(args) == 1:
9         return [dct.get(args[0]) for dct in items if dct.get(args[0]) is not None]
10    else:
11        return [{key : dct.get(key) for key in args if dct.get(key) is not None} for dct in items]
12
13 if __name__ == "__main__":
14     print(field(goods, 'title'))
15     print(field(goods, 'title', 'price'))
```

Результат выполнения

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs % /usr/local/bin/python3 /Users/sofagoraceva/Goryacheva_2025_Labs/Goryacheva_2025_Labs/Lab_2/task_1.py
['Ковер', 'Диван для отдыха']
[{'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха', 'price': 5300}]
- sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs %

Задача 2 (файл gen_random.py)

Необходимо реализовать генератор `gen_random(количество, минимум, максимум)`, который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона. Пример:

`gen_random(5, 1, 3)` должен выдать 5 случайных чисел в диапазоне от 1 до 3, например `2, 2, 3, 2, 1`

Шаблон для реализации генератора:

```
# Пример:
# gen_random(5, 1, 3) должен выдать 5 случайных чисел
# в диапазоне от 1 до 3, например 2, 2, 3, 2, 1
# Hint: типовая реализация занимает 2 строки
def gen_random(num_count, begin, end):
    pass
# Необходимо реализовать генератор
```



Листинг кода

```
1 import random
2
3 def gen_random(num_count, begin, end):
4     return [random.randint(begin, end) for i in range(num_count)]
5
6 if __name__ == "__main__":
7     print(gen_random(5, 2, 10))
```

Результат выполнения

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs % /usr/local/bin/python3 /Users/sofagoraceva/Goryacheva_2025_Labs/Goryacheva_2025_Labs/Lab_2/task_1.py
['Ковер', 'Диван для отдыха']
[{'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха', 'price': 5300}]
- sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs % /usr/local/bin/python3 /Users/sofagoraceva/Goryacheva_2025_Labs/Goryacheva_2025_Labs/Lab_2/task_2.py
[6, 4, 10, 2, 7]
- sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs %

Задача 3 (файл unique.py)

- Необходимо реализовать итератор `Unique(данные)`, который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.
- Конструктор итератора также принимает на вход именованный `bool`-параметр `ignore_case`, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен `False`.
- При реализации необходимо использовать конструкцию `**kwargs`.
- Итератор должен поддерживать работу как со списками, так и с генераторами.
- Итератор не должен модифицировать возвращаемые значения.

Пример:

```
data = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
```

`Unique(data)` будет последовательно возвращать только 1 и 2.

```
data = gen_random(10, 1, 3)
```

`Unique(data)` будет последовательно возвращать только 1, 2 и 3.

```
data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
```

`Unique(data)` будет последовательно возвращать только a, A, b, B.

`Unique(data, ignore_case=True)` будет последовательно возвращать только a, b.

Шаблон для реализации класса-итератора:

```
# Итератор для удаления дубликатов
class Unique(object):
    def __init__(self, items, **kwargs):
        # Нужно реализовать конструктор
        # В качестве ключевого аргумента, конструктор должен принимать bool-параметр ignore_case,
        # в зависимости от значения которого будут считаться одинаковыми строки в разном регистре
        # Например: ignore_case = True, Абв и АВВ – разные строки
        # ignore_case = False, Абв и АВВ – одинаковые строки, одна из которых удалится
        # По-умолчанию ignore_case = False
        pass

    def __next__(self):
        # Нужно реализовать __next__
        pass

    def __iter__(self):
        return self
```

Листинг кода

```

1  from typing import List, Any
2
3  class Unique(object):
4      def __init__(self, items, **kwargs):
5          self._items = iter(items)
6          self._ignore_case = kwargs.get('ignore_case', False)
7          self._seen = set()
8
9      def __next__(self):
10         while True:
11             try:
12                 cur_item = next(self._items)
13             except StopIteration:
14                 raise StopIteration
15
16             key = cur_item
17
18             if self._ignore_case and isinstance(cur_item, str):
19                 key = cur_item.lower()
20
21             if key not in self._seen:
22                 self._seen.add(key)
23                 return cur_item
24
25     def __iter__(self):
26         return self
27
28     def test():
29         data = [1, 4, 3, 2, 2, 1, 10]
30         print([item for item in Unique(data)])
31
32     if __name__ == "__main__":
33         test()

```

Результат выполнения

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
●	sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs % /usr/local/bin/python3 /Users/sofagoraceva/Goryacheva_2025_Labs/Goryacheva_2025_Labs/Lab_2/task_3.py		[1, 4, 3, 2, 10]	
○	sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs %			

Задача 4 (файл sort.py)

Дан массив 1, содержащий положительные и отрицательные числа. Необходимо **одной строкой кода** вывести на экран массив 2, которые содержит значения массива 1, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции sorted. Пример:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
Вывод: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
```



Необходимо решить задачу двумя способами:

1. С использованием lambda-функции.
2. Без использования lambda-функции.

Шаблон реализации:

```
data = [4, -30, 100, -100, 123, 1, 0, -1, -4]

if __name__ == '__main__':
    result = ...
    print(result)

    result_with_lambda = ...
    print(result_with_lambda)
```



Листинг кода

```
1  data = [4, -30, 100, -100, 123, 1, 0, -1, -4]
2
3  def abs_sort_key(x):
4      return abs(x)
5
6  if __name__ == '__main__':
7      print(sorted(data, key = abs_sort_key, reverse=True))
8      print(sorted(data, key = lambda x: abs(x), reverse=True))
```

Результат выполнения

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs % /usr/local/bin/python3 /Users/sofagoraceva/Goryacheva_2025_Labs/Goryacheva_2025_Labs/Lab_2/task_4.py
[123, 100, -100, -30, 4, -4, 1, -1, 0]
[123, 100, -100, -30, 4, -4, 1, -1, 0]
- sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs %

Задача 5 (файл print_result.py)

Необходимо реализовать декоратор print_result, который выводит на экран результат выполнения функции.

- Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.
- Если функция вернула список (list), то значения элементов списка должны выводиться в столбик.
- Если функция вернула словарь (dict), то ключи и значения должны выводиться в столбик через знак равенства.

Шаблон реализации:

```
# Здесь должна быть реализация декоратора

@print_result
def test_1():
    return 1

@print_result
def test_2():
    return 'iu5'

@print_result
def test_3():
    return {'a': 1, 'b': 2}

@print_result
def test_4():
    return [1, 2]

if __name__ == '__main__':
    print('!!!!!!!')
    test_1()
    test_2()
    test_3()
    test_4()
```

Результат выполнения:

```
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2
```

Листинг кода

```

1  from typing import List, Any, Dict
2
3  def print_in_column(data):
4      for el in data:
5          print(el)
6
7  def print_result(func):
8      def wrapper(*args, **kwargs):
9          print("function name:", func.__name__)
10         result = func(*args, **kwargs)
11         if isinstance(result, List):
12             print("result:")
13             print_in_column(result)
14         elif isinstance(result, Dict):
15             print("result:")
16             print_in_column([f'{key} = {value}' for key, value in result.items()])
17         else:
18             print(f"result: {result}")
19         return result
20     return wrapper
21
22 @print_result
23 def test_1():
24     return 1
25
26 @print_result
27 def test_2():
28     return 'iu5'
29
30 @print_result
31 def test_3():
32     return {'a': 1, 'b': 2}
33
34 @print_result
35 def test_4():
36     return [1, 2]
37
38 if __name__ == '__main__':
39     test_1()
40     test_2()
41     test_3()
42     test_4()

```

Результат выполнения

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

● sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs % /usr/local/bin/python3 /Users/sofagoraceva/Goryacheva_2025_Labs/Goryacheva_2025_Labs/Lab_2/task_5.py
function name: test_1
result: 1
function name: test_2
result: iu5
function name: test_3
result:
a = 1
b = 2
function name: test_4
result:
1
2

```


Задача 6 (файл cm_timer.py)

Необходимо написать контекстные менеджеры `cm_timer_1` и `cm_timer_2`, которые считают время работы блока кода и выводят его на экран. Пример:

```
with cm_timer_1():  
    sleep(5.5)
```



После завершения блока кода в консоль должно вывестись `time: 5.5` (реальное время может несколько отличаться).

`cm_timer_1` и `cm_timer_2` реализуют одинаковую функциональность, но должны быть реализованы двумя различными способами (на основе класса и с использованием библиотеки `contextlib`).

Листинг кода

```
1  import time  
2  from contextlib import contextmanager  
3  
4  class cm_timer_1:  
5      def __enter__(self):  
6          self.start = time.perf_counter()  
7          return self  
8      def __exit__(self, exc_type, exc_val, exc_tb):  
9          end = time.perf_counter()  
10         if exc_type is not None:  
11             print("Ошибка в блоке кода")  
12             return False  
13  
14         print(f"time: {end - self.start}")  
15  
16 @contextmanager  
17 def cm_timer_2():  
18     start = time.perf_counter()  
19  
20     try:  
21         yield start  
22     except Exception as e:  
23         print("Ошибка в блоке кода")  
24         raise  
25  
26     print(f"time: {time.perf_counter() - start}")  
27  
28  
29 def test():  
30     print("cm_timer_1")  
31     with cm_timer_1():  
32         time.sleep(5.5)  
33  
34     print("\ncm_timer_2")  
35     with cm_timer_2():  
36         time.sleep(5.5)  
37  
38 if __name__ == "__main__":  
39     test()
```

Результат выполнения

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
• sofagoraceva@MacBook-Air-Sofa Goryacheva_2025_Labs % /usr/local/bin/python3 /Users/sofagoraceva/Goryacheva_2025_Labs/Goryacheva_2025_Labs/Lab_2/task_6.py  
cm_timer_1  
time: 5.5050560829999995  
  
cm_timer_2  
time: 5.501682874999915
```

Задача 7 (файл process_data.py)

- В предыдущих задачах были написаны все требуемые инструменты для работы с данными. Применим их на реальном примере.
- В файле [data_light.json](#) содержится фрагмент списка вакансий.
- Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.
- Необходимо реализовать 4 функции - f1, f2, f3, f4. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора @print_result печатается результат, а контекстный менеджер cm_timer_1 выводит время работы цепочки функций.
- Предполагается, что функции f1, f2, f3 будут реализованы в одну строку. В реализации функции f4 может быть до 3 строк.
- Функция f1 должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр. Используйте наработки из предыдущих задач.
- Функция f2 должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова "программист". Для фильтрации используйте функцию filter.
- Функция f3 должна модифицировать каждый элемент массива, добавив строку "с опытом Python" (все программисты должны быть знакомы с Python). Пример: Программист С# с опытом Python. Для модификации используйте функцию map.
- Функция f4 должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Пример: Программист С# с опытом Python, зарплата 137287 руб. Используйте zip для обработки пары специальность — зарплата.

Шаблон реализации:

```
import json
import sys
# Сделаем другие необходимые импорты

path = None

# Необходимо в переменную path сохранить путь к файлу, который был передан при запуске сценария

with open(path) as f:
    data = json.load(f)

# Далее необходимо реализовать все функции по заданию, заменив `raise NotImplemented`
# Предполагается, что функции f1, f2, f3 будут реализованы в одну строку
# В реализации функции f4 может быть до 3 строк

@print_result
def f1(arg):
    raise NotImplemented

@print_result
def f2(arg):
    raise NotImplemented

@print_result
def f3(arg):
    raise NotImplemented

@print_result
def f4(arg):
    raise NotImplemented

if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))
```

Листинг кода

```
1  import json
2  import sys
3  import os
4
5  sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
6
7
8  import task_1 as field
9  import task_2 as gen_random
10 import task_3 as unique
11 import task_5 as print_result
12 import task_6 as cm_timer
13
14 with open("data.json", 'r', encoding='utf-8') as f:
15     data = json.load(f)
16
17 @print_result.print_result
18 def f1(data):
19     job_names = list(field.field(data, "job-name"))
20     unique_jobs = unique.Unique(job_names, ignore_case=True)
21     return sorted(unique_jobs, key=lambda s: s.lower())
22
23
24 @print_result.print_result
25 def f2(data):
26     return list(filter(lambda s: "программист" in s.lower(), data))
27
28
29 @print_result.print_result
30 def f3(data):
31     return list(map(lambda s: f"{s} с опытом Python", data))
32
33
34 @print_result.print_result
35 def f4(data):
36     salaries = list(gen_random.gen_random(len(data), 100000, 200000))
37     result = []
38     for job, salary in zip(data, salaries):
39         result.append(f"{job}, зарплата {salary} руб.")
40     return result
41
42
43 if __name__ == "__main__":
44     with cm_timer.cm_timer_1():
45         f4(f3(f2(f1(data))))
```

Результат выполнения

Вывод результата выполнения слишком большой, его можно посмотреть в репозитории по ссылке: [результат_выполнения](#)